Final Project APMA4302
Fred Angelo Garcia (fbg2107)
May 14, 2026

1 Introduction

Cosmological simulations aim to model how our Universe's large scale structer – with galaxies as the main building blocks – came about. They are ran using in a (radiative magneto)-hydrodynamics codes: the gas in our universe is a compressible fluid. What primarily drives the evolution of this fluid are gravity and astrophysical processes (e.g., star formation, black hole accretion, etc.). A notable example of a cosmological simulation code that is used to run large scale cosmological simulations is called `AREPO` [Weinberger et al., 2020], which is a moving mesh code that solves the hydrodynamics equations on an unstructured Voronoi mesh. The astrophysical models that are implemented in the code and used to run large scale cosmological simulations is called IllustrisTNG [Nelson et al., 2019]. The hydrodynamics code and the astrophysical models are two separate things. The hydrodynamics code is the numerical method that solves the equations of fluid dynamics, while the astrophysical models are the subgrid physics that are implemented in the code to model the unresolved physics (e.g., how black holes accrete matter, how stars exploding as supernovae affect the surrounding gas, etc.).

In this project, we will take the time to step back and look at the fundamentals of finite volume methods for solving the hydrodynamics equations that we use daily. These will also function as a personal reference for me, if we ever need to understand what's going on under the hood of the hydrodynamics code that we use. Here, we will implement a simple (parallelizable) finite volume solver in PETSc to solve simple 1D problems for which there are analytic solutions (e.g., the Sod shock tube problem), test convergence, and analyze different numerical tricks, such as slope limiters, to ensure numerical stability especially in the presence of shocks, a key astrophysical process relvant in supernovea and blackhole jet ejecta.

2 Methods

Recall, most of these have been mentioned in the proposal. We start with the equations of hydrodynamics. Which are hyperbolic partial differential equations that describe the conservation of mass, momentum, and energy in a fluid. Euler usually suffices, since the universe's scales are so large that viscosity and thermal conduction are negligible to warrant the use of the Navier-Stokes equations.

The equations can be written in conservative form (with source terms for gravity and astrophysical processes, which we will ignore for this project) as

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{v}) = 0, \tag{1}$$

$$\frac{\partial (\rho \mathbf{v})}{\partial t} + \nabla \cdot (\rho \mathbf{U}\mathbf{U}) + \nabla p = 0, \tag{2}$$

$$\frac{\partial \rho E}{\partial t} + \nabla \cdot (\rho E \mathbf{U} + p\mathbf{U}) = 0. \tag{3}$$

where ρ is the mass density, $\mathbf{U} = u\hat{x} + v\hat{y}$ is the velocity vector, p is the pressure, and E is the total energy per unit mass[1]. The specific energy e is related to the total energy by $E = e + \frac{1}{2}|\mathbf{U}|^2$. The system is closed by an equation of state, which for an ideal gas is typically expresed as $p = (\gamma - 1)\rho e$, where γ is the adiabatic index (e.g., $\gamma = 5/3$ for a monatomic ideal gas) and a given sound speed c_s is given by $c_s = \sqrt{\gamma p/\rho}$.

[1]Note, we will use much of the notation here and in the code as in `github.com/Open-Astrophysics-Bookshelf/numerical_exercises` an intro to numerical methods textbook by M. Zingale at Stony Brook

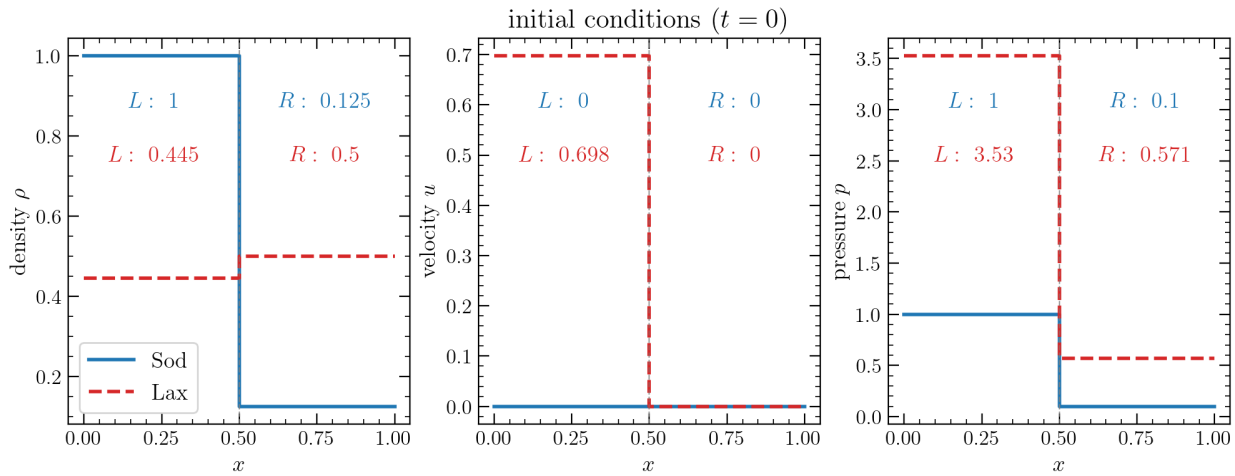

Figure 1: Initial conditions for the Sod shock tube problem (solid blue) and the Lax shock tube problem (dashed red).

2.1 1D Euler Equations

In this project we restrict to one spatial dimension. This simplifies the equations to

$$\frac{\partial \rho}{\partial t} + \frac{\partial (\rho u)}{\partial x} = 0, \tag{4}$$

$$\frac{\partial (\rho u)}{\partial t} + \frac{\partial (\rho u u + p)}{\partial x} = 0, \tag{5}$$

$$\frac{\partial \rho E}{\partial t} + \frac{\partial (u(\rho E + p))}{\partial x} = 0. \tag{6}$$

In this conservative form $\mathcal{U}_t + \mathbf{F}(\mathcal{U})_x = 0$, we can write the system as a pair of vector quatities

$$\mathcal{U} = \begin{bmatrix} \rho \\ \rho u \\ \rho E \end{bmatrix}, \qquad \mathbf{F}(\mathcal{U}) = \begin{bmatrix} \rho u \\ \rho u u + p \\ u(\rho E + p) \end{bmatrix}. \tag{7}$$

Expressed this way, these are the conserved variables: mass density ρ, momentum density ρu, and energy density ρE. The fluxes are the mass flux ρu, the momentum flux $\rho u u + p$, and the energy flux $u(\rho E + p)$. These are PDEs, which means information propagates via waves, there is a finite speed of propagation, and the solution can develop discontinuities (e.g., shocks) even if the initial conditions are smooth.

2.2 Riemann Problem Setup

For Professor: tl;dr we did MUSCL-HLLC in PetSc.

Riemann solvers are good at these hyperbolic systems problems like those set up in Figure 1. Such problems are the Sod shock tube problem and the Lax shock tube problem, which are two classic test problems for hydrodynamics codes. The initial conditions are piecewise constant, with a discontinuity at $x = 0.5$. For Sod, there is a density and pressure jump, but the velocity is zero everywhere. For Lax, there is a density, velocity, and pressure jump.

The solution to these problems consists of three nonlinear wave structures: a left-going rarefaction wave, a contact discontinuity, and a right-going shock wave. These waves propagate according to the characteristic structure of the Euler equations, whose eigenvalues are $u - c$, u, and $u + c$. The contact discontinuity propagates with the fluid velocity in the intermediate (star) region, while the shock and rarefaction propagate at nonlinear wave speeds determined by the Rankine–Hugoniot conditions and characteristic fan structure.

Unlike linear advection problems, these solutions do not simply translate while preserving shape; instead, nonlinear effects generate shocks and rarefaction fans, making them non-trivial tests for numerical methods.

As a side note, in a numerical finite-volume context, the left and right states at each interface are often denoted $\mathcal{U}_L = \mathcal{U}_i$ and $\mathcal{U}_R = \mathcal{U}_{i+1}$, respectively.

2.3 Finite Volume Discretization

To actually solve these equations, we need to discretize them. We lay down a grid for $[0, 1]$ into N uniform cells of width $\Delta x = 1/N$. Each cell i stores the volume-averaged conserved variables $\mathcal{U}_i$; i.e., each cell has uniform properties. Integrating the conservation law over cell i yields the exact update relation, where n is the time step index,

$$\mathcal{U}_i^{n+1} = \mathcal{U}_i^n - \frac{\Delta t}{\Delta x}\left(\mathbf{F}_{i+1/2}^n - \mathbf{F}_{i-1/2}^n\right), \quad (8)$$

where $\mathbf{F}_{i\pm1/2}$ is the numerical flux at the left and right faces of cell i, which must be approximated. The simplest choice is the Lax-Friedrichs (LF) flux, which we include as a first-order baseline for comparison,

$$\begin{aligned}\mathbf{F}_{i+1/2}^{\mathrm{LF}} = &\frac{1}{2}(\mathbf{F}(\mathcal{U}_i) + \mathbf{F}(\mathcal{U}_{i+1})) \\ &- \frac{\Delta x}{2\Delta t}(\mathcal{U}_{i+1} - \mathcal{U}_i)\,,\end{aligned} \quad (9)$$

which is first-order accurate and requires no knowledge of the local wave structure. Breaking down the LF equation, the term:

$$\frac{1}{2}[\mathbf{F}(\mathcal{U}_i) + \mathbf{F}(\mathcal{U}_{i+1})] \quad (10)$$

is a central average term quantifying the average flux coming from the left and right cells (more on this later when we talk about slope limiters and reconstruction). The term

$$-\frac{\Delta x}{2\Delta t}(\mathcal{U}_{i+1} - \mathcal{U}_i) \quad (11)$$

is a diffusive term that stabilizes the solution by sort of smearing out the solution and preventing unphysical oscillations near shocks. In particular, there is a stability condition, called the Courant-Friedrichs-Lewy (CFL) condition, which state the step size

$$\Delta t = \nu\,\frac{\Delta x}{\max\limits_i\left(|u_i| + c_{s,i}\right)}, \qquad c_s = \sqrt{\frac{\gamma p}{\rho}}, \quad (12)$$

[Courant et al., 1928] with CFL number $\nu < 1$ and the local sound speed c_s.

In this work, all production runs use the HLLC (Harten-Lax-van Leer-Contact) approximate Riemann solver instead. The overall finite-volume framework is identical for LF and HLLC, but the flux equations themselves are different:

$$\mathbf{F}_{i+1/2}^{\mathrm{HLLC}} = \begin{cases}\mathbf{F}(\mathcal{U}_L), & S_L > 0, \\ \mathbf{F}^*, & S_L \le 0 \le S_R, \\ \mathbf{F}(\mathcal{U}_R), & S_R < 0,\end{cases} \quad (13)$$

where S_L and S_R are estimates of the left and right wave speeds, and $\mathbf{F}^*$ is the flux in the star region between the waves, which is computed by enforcing the Rankine–Hugoniot jump conditions across the waves, which ensures the correct shock speeds and contact discontinuity propagation. In short, these are stament of convervation of mass, energy, and momentum across the shock Krehl [2015].

2.4 Reconstruction

A first-order scheme feeds the raw cell-averaged values directly into the flux function, i.e. $\mathbf{F}(\mathcal{U}_i)$ is evaluated at the cell centre. This piecewise-constant reconstruction is only first-order accurate. To achieve higher-order accuracy, we need to do better.

Reconstruction is the process of fitting a function to the cell-averaged values in order to obtain more accurate estimates of the solution at the cell interfaces. A piecewise-constant reconstruction

introduces excessive numerical diffusion. The Monotone Upstream-centred Schemes for Conservation Laws (MUSCL) framework increases accuracy by prescribing some non-linear interpolation of the cell-averaged values to obtain left and right states at the cell interfaces, which are then used as inputs to the numerical flux function.

Reconstruction is performed on the primitive variables $\mathbf{W} = \{\rho, u, p\}$. For each scalar component $q \in \mathbf{W}$, the left and right interface states at the cell face $i + \frac{1}{2}$ are constructed as

$$q^L_{i+1/2} = q_i + \tfrac{1}{2}\,\phi(r_i)\,(q_i - q_{i-1}), \tag{14}$$

$$q^R_{i+1/2} = q_{i+1} - \tfrac{1}{2}\,\phi(r_{i+1})\,(q_{i+2} - q_{i+1}) \tag{15}$$

where the ratio of successive differences is defined by

$$r_i = \frac{q_i - q_{i-1}}{q_{i+1} - q_i}. \tag{16}$$

This ratio measures local smoothness: $r_i \approx 1$ in smooth regions, while large or small values indicate steep gradients or discontinuities.

2.5 Slope Limiters

Now for some numerical wizzardry.

The limiter function $\phi(r)$ suppresses spurious oscillations near shocks while retaining second-order accuracy in smooth regions. Common limiters satisfying the constraints of Sweby [1984] include

$$\phi_{\text{MinMod}}(r) = \max\big(0, \min(1, r)\big), \tag{17}$$

$$\phi_{\text{van Leer}}(r) = \frac{r + |r|}{1 + |r|}, \tag{18}$$

$$\phi_{\text{Superbee}}(r) = \max\big(0, \min(2r, 1), \min(r, 2)\big). \tag{19}$$

Setting $\phi = 0$ reduces the scheme to first-order piecewise-constant reconstruction. Note, other limiters can be found here: `wikipedia.org/wiki/Flux_limiter`.

The reconstructed interface states are then used as inputs to the numerical flux function in the finite-volume update we initially wrote down in Equation 8, which is now modified to

$$\begin{aligned}\mathcal{U}_i^{n+1} = \mathcal{U}_i^n - \frac{\Delta t}{\Delta x}\Big(&\mathbf{F}^n_{i+1/2}\Big(\mathcal{U}^L_{i+1/2}, \mathcal{U}^R_{i+1/2}\Big) \\ &- \mathbf{F}^n_{i-1/2}\Big(\mathcal{U}^L_{i-1/2}, \mathcal{U}^R_{i-1/2}\Big)\Big),\end{aligned} \tag{20}$$

where the numerical flux $\mathbf{F}^n_{i+1/2}$ is obtained by passing the reconstructed states $\mathcal{U}^L_{i+1/2}$ and $\mathcal{U}^R_{i+1/2}$ to a Riemann solver. In this project that solver is HLLC, which approximates the intercell flux by accounting for the three-wave structure of the Euler equations.

3 Code Implementation

The bulk of the numerics is inspired by the Philip Mocz article, which is a Python implementation of a finite volume solver for the 1D Euler equations. we have implemented the same algorithm in C using PETSc, which can parallelize the computation across multiple processors.

The main steps of the algorithm are as follows:

1. Compute CFL condition:the local maximum signal speed $\max_i(|u_i| + c_{s,i})$ over owned cells, then `MPI_Allreduce` to obtain the global maximum across all ranks. The time step is $\Delta t = \nu\,\Delta x/\lambda_{\max}$ with $\nu = 0.4$.

2. Call `DMGlobalToLocalBegin/End` to fill ghost cells from neighbouring ranks, making cells $i - 2$ through $i + N_{\text{local}} + 1$ fully populated (two ghost layers on each side), where N_{local} is the number of cells in the rank.[2]

[2]Ghost cells are extra cells stored on each rank that hold copies of the data from neighboring ranks, allowing

3. At the physical domain boundaries, copy the innermost owned cell into the ghost layers (transmissive outflow): $\mathcal{U}_{-1} = \mathcal{U}_{-2} = \mathcal{U}_0$ on the left, and symmetrically on the right.

4. Convert $\mathcal{U} \to \mathbf{W} = (\rho, u, p)^T$ in every cell including ghosts, using the ideal-gas EOS $p = (\gamma - 1)(E - \frac{1}{2}\rho u^2)$.

5. For each face $i+\frac{1}{2}$ in the owned stencil, apply the slope limiter to obtain $(q^L_{i+1/2}, q^R_{i+1/2})$ for each primitive $q \in \{\rho, u, p\}$, convert to conserved states, then call `flux_hllc` [Toro et al., 1994] to obtain $\mathbf{F}_{i+1/2}$.

6. Apply Equation 8: $\mathcal{U}^{n+1}_i = \mathcal{U}^n_i - (\Delta t/\Delta x)(\mathbf{F}_{i+1/2} - \mathbf{F}_{i-1/2})$, writing the result into the global vector.

The meat of the code is in `euler1d.c`, contained in the `src` directory. Just make the files, and the args are as follows:

```
Usage: mpirun -n <num_procs> ./euler1d [options]
Options: (default *)
  "  -da_grid_x N        number of cells (default 400)"
    "  -cfl C              CFL number (default 0.4)"
    "  -t_end T            end time (default 0.2)"
    "  -output_freq F      save every F steps (default 10)"
    "  -riemann_type TYPE  lf | hllc*"
    "  -limiter_type TYPE  none | minmod* | vanleer | superbee"
    "  -problem TYPE       sod* | lax | vacuum "
    "  -gamma G            adiabatic index (default 5/3)";
```

`run_sims.sh` runs all the required runs to produce the plots, of which can all be redrawn by doing `run_plots.sh`.

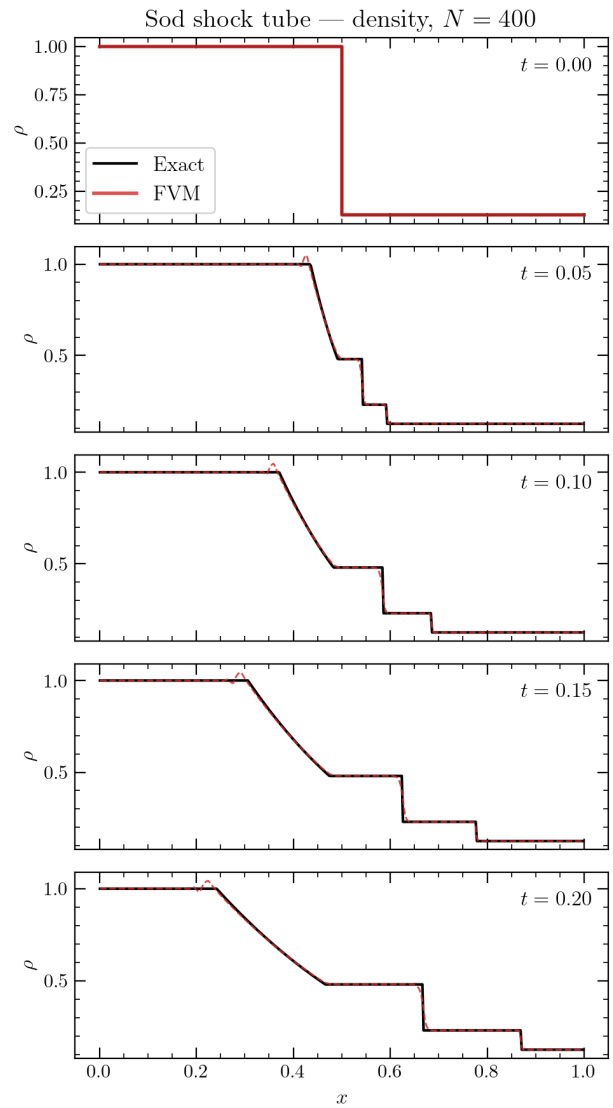

Figure 2: Time evolution of the solution for the Sod shock tube problem with ran with `mpiexec -np 4 ./euler1d -da_grid_x 400 -problem sod -t_end T -riemann_type hllc -limiter_type minmod -output_freq 99999 -gamma 1.6667`. The solution consists of a left-going rarefaction wave, a contact discontinuity, and a right-going shock wave.
Where we modified T end time to show evolution.

4 Results

Using known analytic solutions to the Sod and Lax shock tube problems, we look at performance, convergence, and the effect of different

for local computations that require information from adjacent cells without needing to communicate during the computation. They provide stencial access to neighboring cell values. Thankfully PETSc's `DMDA` object manages this automatically: each rank allocates storage for its owned cells plus a layer of ghost cells on each side, and `DMGlobalToLocal` fills those ghost cells by communicating with adjacent ranks before each flux evaluation.

slope limiters on the solution.

4.1 Solution Structure: Sod and Lax

First, we look at the time evolution of the solution for the Sod shock tube problem in Figure 2, which shows the density computed using HLLC+MinMod evolved from the initial conditions to $t = 0.2$. The solutions look reasonable when compared with the analytic solution (from Phil Mocz's code), with the expected wave structure of a left-going rarefaction wave, a contact discontinuity, and a right-going shock wave. There are bumps near the shock and contact discontinuity, which are expected since these are regions of strong gradients where the solution is not smooth.

we then ran it through the Lax shock tube problem, which has a more extreme initial condition with a higher velocity jump and a lower pressure on the right side. The solution is shown in Figure 3, which shows a single snapshot at $t = 0.13$. The solution also looks reasonable, with the expected wave structure of a left-going rarefaction wave, a contact discontinuity, and a right-going shock wave. However, there are more pronounced bumps (like a sinosoid shape) near the shock and contact discontinuity compared to the Sod problem, which is expected since the Lax problem has stronger gradients and is more challenging to solve numerically as per the initial conditions shown in Figure 1.

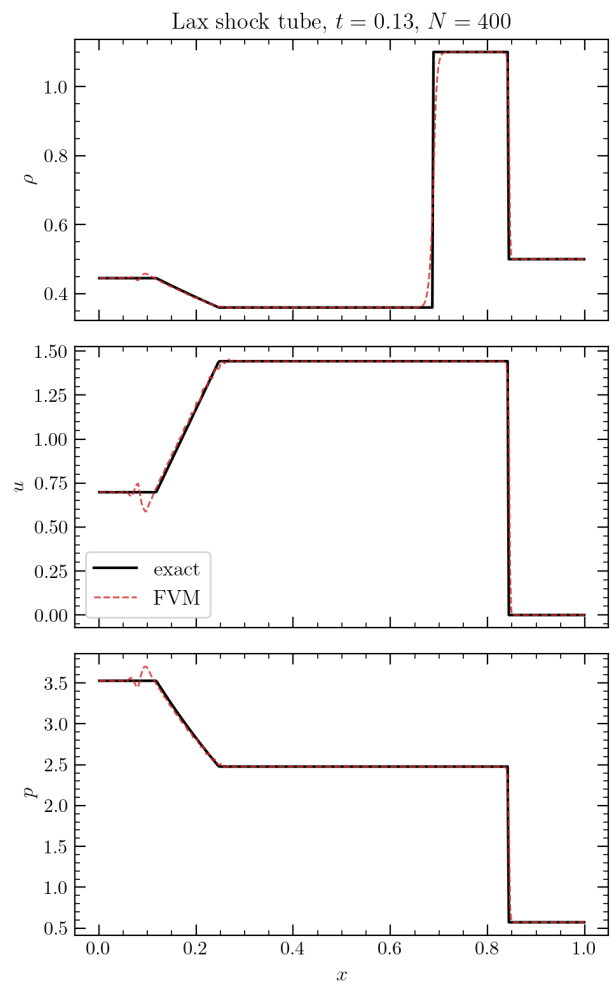

Figure 3: Solution for the Lax shock tube problem at $t = 0.13$, using the initial conditions shown in the red dashed line of Figure 1.

4.2 Limiter Comparison

We now compare various slope limiters for the Sod shock tube problem. The results are shown in Figure 4, which shows the density, velocity and pressure computed using HLLC with different slope limiters at $t = 0.2$. The solution with no limiter (i.e., piecewise-constant reconstruction) is very diffusive, with the shock and contact discontinuity being smeared out over many cells (blue). The MinMod limiter (orange) produces

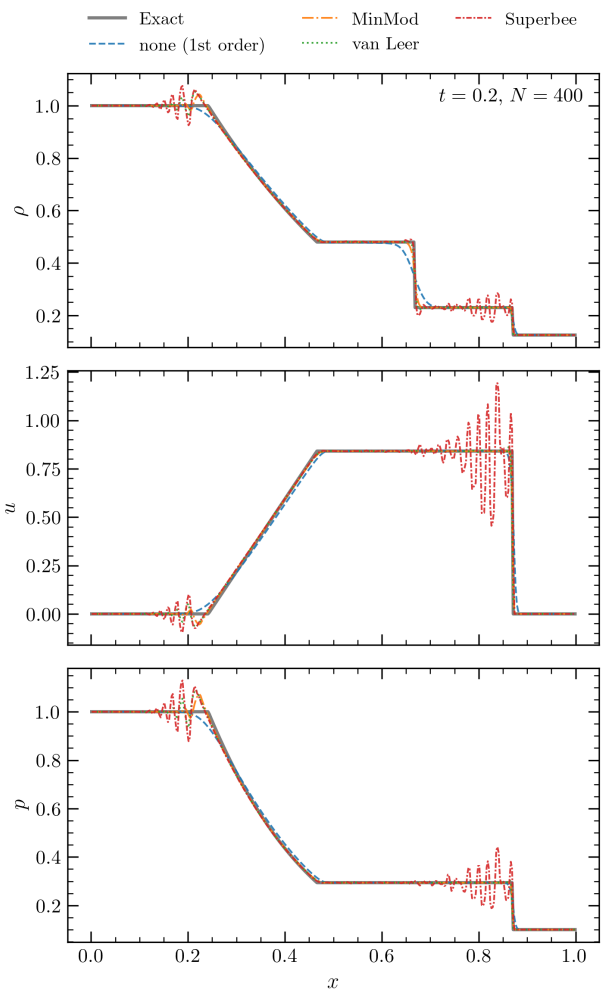

Figure 4: Comparison of different slope limiters for the Sod shock tube problem at $t = 0.2$.

a sharper solution, but there are still some oscillations near the shock and contact discontinuity. The van Leer limiter (green) produces a sharper solution than MinMod, but there are more pronounced oscillations near the shock and contact discontinuity. The Superbee limiter (red) produces the sharpest solution. For example, if we look near $x = 6.5$, the drop in density is more pronounced. But, there are significant oscillations near the shock and contact discontinuity in other places, which can be unphysical.

Note, these oscillations still satisfy the total variation diminishing (TVD) condition, which means they do not grow in time and are not unbounded [Harten, 1983]. However, they can still be undesirable since they can lead to negative densities or pressures, which are unphysical.

Overall, the choice of slope limiter is a tradeoff between accuracy and stability, with more aggressive limiters producing sharper solutions but also more oscillations. Personally, no limiter (blue, which is piecewise-constant) looks an attractive soluton. However, these are not ideal for capturing shocks. Superbee limiter looks like a very sharp solution, which is ideal for capturing shocks but the oscillations look too much, especially in velocity. The MinMod and van Leer limiters look OK, apart from the small oscillations – these look better.

Looking at some papers in my field, another hydro code (AMR in this case called `RAMSES` [Teyssier, 2002]) uses the MinMod limiter. So this results makes sense.

4.3 Convergence

Finally, we do a quick convergence check on the L^1 error in density, defined as

$$\|\rho - \rho_{\text{exact}}\|_1 = \sum_{i=1}^{N} |\rho_i - \rho_{\text{exact}}(x_i)|\, \Delta x, \quad (21)$$

where ρ_i is the numerical cell-average and $\rho_{\text{exact}}(x_i)$ is the exact Riemann solution evaluated at the

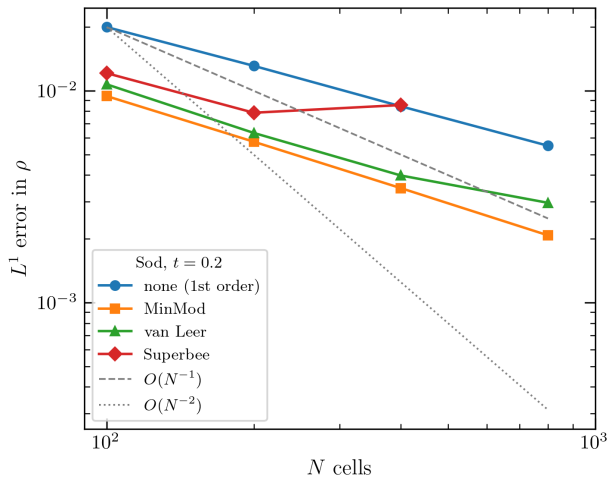

Figure 5: L^1 error in density vs. number of cells for the Sod shock tube at $t = 0.2$, for four slope limiters using HLLC. Dashed and dotted grey lines show $O(N^{-1})$ and $O(N^{-2})$ reference slopes, respectively.

cell centre. The results are shown in Figure 5 for all four slope limiters at $N = 100, 200, 400, 800$ for the Sod shock tube at $t = 0.2$.

All four limiters converge, but at a rate *slower* than first order $\sim O(N^{-0.7})$. This is a little worrying, but we guess the contact discontinuity in the Sod solution is the dominant error source; its width is $O(N^{-1})$ cells but its contribution to the L^1 norm decreases only as $O(N^{-1})$, independent of limiter, pulling the measured slope below one. Away from the discontinuity the scheme is locally second-order, but the global rate is set by the worst region. so that's why it is a shallower slope than $O(N^{-1})$.

The piecewise-constant (no limiter) run tracks the other three closely because the contact discontinuity error dominates over the additional diffusion introduced by dropping the MUSCL slopes; its slightly higher error level reflects the extra smearing everywhere else. MinMod and van Leer produce nearly identical error magnitudes, with van Leer marginally lower. Superbee is the most accurate in the smooth rarefaction fan but shows slightly elevated errors near the contact due to its aggressive compression, consistent with the oscillations seen in Figure 4, so much so that it has the highest overall error at $N = 800$.

5 Discussion and Conclusion

This project allowed to take a step back and look at the fundamentals of finite volume methods for solving the hydrodynamics equations that we use daily. we implemented a simple (parallelizable) finite volume solver in PETSc to solve simple 1D problems for which there are analytic solutions (e.g., the Sod shock tube problem), tested convergence, and analyzed different numerical tricks, such as slope limiters, to ensure numerical stability at shoscks.

Slop limiters are a key ingredient in MUSCL-type schemes, which are widely used in astrophysical hydrodynamics codes. Here, we demonstrate the minmod is the best out of the ones we tested. Nonetheless, the MUSCL-HLLC solvers we explore here are just one class of Riemann solvers, and there are many other types of solvers that can be used to solve the Euler equations, which we hope to explore in the future.

References

Richard Courant, Kurt Friedrichs, and Hans Lewy. Über die partiellen differenzengleichungen der mathematischen physik. *Mathematische Annalen*, 100(1):32–74, 1928.

Ami Harten. High Resolution Schemes for Hyperbolic Conservation Laws. *Journal of Computational Physics*, 49(3):357–393, March 1983. doi: 10.1016/0021-9991(83)90136-5.

Peter O. K. Krehl. The classical Rankine-Hugoniot jump conditions, an important cornerstone of modern shock wave physics: ideal

assumptions vs. reality. *European Physical Journal H*, 40(2), March 2015. doi: 10.1140/epjh/e2015-50010-4.

Dylan Nelson, Volker Springel, Annalisa Pillepich, Vicente Rodriguez-Gomez, Paul Torrey, Shy Genel, Mark Vogelsberger, Ruediger Pakmor, Federico Marinacci, Rainer Weinberger, Luke Kelley, Mark Lovell, Benedikt Diemer, and Lars Hernquist. The IllustrisTNG simulations: public data release. *Computational Astrophysics and Cosmology*, 6(1):2, May 2019. doi: 10.1186/s40668-019-0028-x.

P. K. Sweby. High Resolution Schemes Using Flux Limiters for Hyperbolic Conservation Laws. *SIAM Journal on Numerical Analysis*, 21(5):995–1011, October 1984. doi: 10.1137/0721062.

R. Teyssier. Cosmological hydrodynamics with adaptive mesh refinement. A new high resolution code called RAMSES. *Astronomy & Astrophysics*, 385:337–364, April 2002. doi: 10.1051/0004-6361:20011817.

E. F. Toro, M. Spruce, and W. Speares. Restoration of the contact surface in the HLL-Riemann solver. *Shock Waves*, 4(1):25–34, July 1994. doi: 10.1007/BF01414629.

Rainer Weinberger, Volker Springel, and Rüdiger Pakmor. The AREPO Public Code Release. *ApJ*, 248(2):32, June 2020. doi: 10.3847/1538-4365/ab908c.